ناطق · NAATIQ — DOCUMENTATION

Build Prompt Log

Every instruction the human gave, verbatim and in order, with what each one produced

Team · Karam Judeh · Rahma Al Sharif · Abderrahim Laghmari

What this is

Document 05 records the **product prompts** — the two system prompts that run in production every time a worker sends a message. Document 06 explains the **working method**.

This document is the raw record: **every prompt Karam gave during the build, reproduced verbatim, in the order given.** Spelling and phrasing are unedited — this is what was actually typed, not a cleaned-up version. Only credentials have been redacted.

It is included because the prompts *are* the build log. Naatiq was specified, corrected, debugged and shipped through this sequence of messages, and the sequence is more informative about how the product came to exist than any retrospective narrative.

Phase 1 — Framing

Prompt 1

> "please read the md file and tell me what you will do, if you have any questions please state them before implementing. do not do anything extra or connect to any app without running it by me first."

Produced: the entire working contract. Two rules that held for the whole session — plan before implementing, and no unrequested side effects. Every external connection that followed was proposed and approved first.

Decisions given in reply to structured questions: *Full end-to-end build · Nothing yet — greenfield · Yes, read & write Supabase access · New Wathefti project.*

Prompt 2

> "also github repo is https://github.com/Kjudeh/Kanz-Hackathon.git please connect to it and have everything pushed there"

Produced: the repository connection. Everything from this point was committed and pushed.

Prompt 3

> "I'll paste a GitHub token"

followed by two fine-grained personal access tokens *(redacted)*, both of which returned 403, then:

> "Classic token instead"

(classic tokens redacted), then:

> "continue please"

Produced: working push access, after two failed token types. See document 06.

Phase 2 — Building the core

Decisions: *Build Phase 1 + 2 code now* → the schema and the WhatsApp agent. *Build Phase 3: the CV* → the bilingual template and PDF pipeline. *Build Phase 4: the dashboard* → the impact dashboard. *Phase 5 features → Arabic voice replies (TTS). Agent + CV + docs.*

Phase 3 — Brand and identity

Prompt 4

> "i added a file 2026-07-18 13.54.40.jpg please use that as the logo for the dashboard, and use the colours in it for as a moodboard for the dashboard, we also changes the name of the app to ناطق"

Produced: the entire visual system, from one image. The palette was sampled directly from the logo — navy `#001737`, blue `#0E52E5`, emerald `#09A385` — and the rename from *Wathefti* to *Naatiq* propagated through the agent persona, the CV footer, the dashboard, the web pages and these documents.

Prompt 5

> "i need the dashboard in arabic with english as an option, there is an error that say could not load live data: failed to fetch"

Produced: the Arabic-first RTL dashboard with an English toggle, and the fix for the data-loading failure.

Prompt 6

> "if the dahsboard is live why is it showing demo data?"

Produced: the correct diagnosis of the fallback behaviour, and the fix that made live data actually load.

Prompt 7

> "also did you update the cv template?"

Prompt 8

> "and the sample cv please"

Prompt 9

> "can you add the logo to the footer of the CV?"

Produced: the rebranded CV template, a rendered sample to check it against, and the Naatiq mark in the CV footer.

Phase 4 — Going live

Prompt 10

> "perfect now what api tokens and secrets do you want from me?"

Produced: the full list of required secrets, which became the configuration section of the runbook.

Prompt 11 *(with a screenshot of five configured secrets)*

> "these are added. do we need gotenberg? on my beautistaff, we generate the pdfs without a third party tool"

Produced: the most useful challenge of the build. The answer was not a defence: PDFs can be generated without a third-party service, but **Arabic** PDFs are a different problem — contextual letter shaping and bidirectional layout are things most PDF libraries get wrong. Gotenberg wraps headless Chromium and is self-hosted, so it is infrastructure rather than a vendor. The reasoning is recorded in document 07.

Decision: *Deploy Gotenberg.*

Prompt 12

> "how do i set this, gotenberg ready"

Produced: the Railway deployment walkthrough.

Prompt 13

> "everything is set; all the keys and twilio webhook"

Phase 5 — Debugging with a real phone

Every prompt in this phase is a bug report from someone actually using the product. None of these would have surfaced from reading code.

Prompt 14

> "sent a voice message im receiving nothing, please fix"

Produced: the turning point of the build. Supabase's log API exposes request lines but not `console.error` output, so this was undiagnosable by inspection. The response was to build the **self-test diagnostic endpoint**, which then found four production bugs in minutes each — the rejected `temperature` parameter, the Twilio channel error, the missing URL scheme on Gotenberg, and the ElevenLabs plan restriction.

Prompt 15

> "i also added elevenlabs key"

Prompt 16

> "can you walk me through the set up of elevenlabs?"

Prompt 17

> "done, please configure, and clean the database from the all the candidates and demo ones, so i can start testing."

Produced: a clean database for real testing, and the reset procedure now in the runbook.

Prompt 18

> "perfect now waiting for the CV to be sent, i am receiving correct replies and its understanding me correctly, but i am not receiving replies as voice notes only written ones."

Produced: confirmation that the interview itself was working correctly in live use — the single most important validation in the project — plus the ElevenLabs 402 investigation.

Prompt 19

> "i chnaged the id to a free one, can you test? on the dashboard it shows that i have completed the conversation but no link to the pdf"

Produced: `cv_pdf_path` exposed through `dashboard_profiles`, and CV links on the dashboard.

Decision: *"full access."* — chosen after being warned that a public `cvs` bucket means CV PDFs containing names and phone numbers are reachable by anyone with the URL. A deliberate, informed hackathon trade-off, documented with its production fix in document 08.

Prompt 20

> "i changed the voice to the recommended one from you, please test"

Produced: the third 402. Only at this point was the real cause established by reading the documentation: the ElevenLabs voice library is unavailable over the API on free plans, so no voice ID would have worked. Two recommendations had been wrong before that. See document 06.

Decision: *Cut voice replies for now* — the feature stayed deployed behind a flag rather than being removed.

Phase 6 — Widening the surface

Prompt 21

> "i need you to create a landing page with the whatsapp number, this landing page should have the same colours as the logo, the logo big and undertsandable, a call to action button, how many CVs generated, a page for clients to find the CVs and employ, how many employments have happened. who we are targeting which candidates, which jobs."

Produced: the landing page and the employer portal — and a schema change. There was no honest way to display an employment count, so the `placements` table was added rather than inventing a number.

Prompt 22

> "add on the landing page how it works also"

Prompt 23

> "perfect thank you we will change the design a bit later. now the most important part i need you to create a diagram that shows the whole process cvg or png file"

Produced: `docs/naatiq-architecture.svg` and `.png` — the three-band system diagram.

Phase 7 — Documentation

Prompt 24

> "create a documentation file, and create pdfs of all the prompts used, tools used, an updated prd, target audience, technical document, and see what else missing from documentation that we need to add."

Produced: this documentation set. The "see what else is missing" clause is what added the six documents beyond the five requested — the index, security and privacy, the setup and runbook, testing and limitations, and the demo script.

Decisions: *Both, as separate docs* — product prompts and build prompts documented separately. *Separate PDF per document.*

Prompt 25

> "missing from the documentation is the prompts used by me to make you build all of this."

Produced: this document. Document 06 had quoted a selection of the build prompts inside a narrative; what was missing was the complete verbatim record in order. That gap was correctly identified.

What the sequence shows

Twenty-five prompts and roughly fifteen structured decisions built the entire product — two Edge Functions, two production prompts, four migrations with a schema-level privacy model, a bilingual CV template, three front-end surfaces, a self-hosted PDF service, an architecture diagram and thirteen documents.

Four patterns are visible in the record:

The first prompt did the most work. "Tell me what you will do... do not do anything extra" established plan-then-build and no-surprises, and neither rule needed restating.

Direction came as short decisions, not specifications. Scope was set through brief multiple-choice answers rather than a written spec. Faster to give, and fewer misunderstandings than a document would have produced.

The bug reports came from use, not inspection. "sent a voice message im receiving nothing", "no link to the pdf", "why is it showing demo data" — every significant defect was found by a person holding a phone.

Challenges improved the product. "do we need gotenberg?" forced the Arabic-rendering rationale to be articulated rather than assumed, and that reasoning is now the most defensible technical decision in the stack.